

Application - Routes and Pages

The App Router surface: nine API endpoints and four pages

EdgePilot AI - Team 15
Documentation set, 23 August 2026
17 documents in this volume

src/app/ - routes and pages

Work packages: product UI (pages) and integration (the API surface), with individual routes belonging to their modules.

What this folder is

The Next.js App Router. Every folder here is a URL. Nothing here holds business logic: a route parses, dispatches and picks a status code, and a page mounts a component.

What happened here

The clearest single number in this project is here. `dashboard/page.tsx` went from **692 lines to 15**. That is not deletion - the behaviour moved into `src/components/dashboard/`, where it can be tested without a request. A route file that is 692 lines long is a route file that cannot be unit tested.

`globals.css` went from 418 lines to 27 for the same reason: the dashboard's styling belongs to the dashboard, scoped under `.epd`, not to every page in the application.

`auth/` and `api/auth/` were removed - there are no accounts. A new page, `evidence/`, was added. `vision-benchmark/page.tsx` grew from 73 to 244 lines to carry the dataset upload.

Files

File	What it is
<code>layout.tsx</code>	23 lines. The root layout and metadata
<code>page.tsx</code>	37 lines. The home page
<code>globals.css</code>	28 lines. Genuinely global styling only

Subfolders

Subfolder	Route
<code>api/</code>	<code>/api/*</code> - the whole HTTP API
<code>dashboard/</code>	<code>/dashboard</code> - the benchmark journey
<code>vision-benchmark/</code>	<code>/vision-benchmark</code>
<code>evidence/</code>	<code>/evidence</code> - artefacts, rendered

Connected folders

- `src/components/` - what these pages mount.
- `src/modules/` - what these routes dispatch to.
- `public/` - why a static `index.html` had to be removed from there.

Referenced from

These folders point here. Each link below resolves in both directions.

- [src/](#)

src/app/api/ - the HTTP API

Work package: integration and API surface.

What this folder is

Route handlers. One version namespace, v1.

What happened here

api/auth/ was removed along with the auth pages. Everything else lives one level down.

Subfolders

Subfolder	What it is
v1/	The versioned API - nine endpoints

Connected folders

- src/app/api/v1/ - the endpoints.
- docs/internal/ - benchmark-api.md is the authoritative wire contract.

Referenced from

These folders point here. Each link below resolves in both directions.

- src/app/

src/app/dashboard/ - the benchmark dashboard route

Work package: product UI and benchmark dashboard.

What this folder is

The /dashboard route: a page, a loading state, an error state, and the module's stylesheet.

What happened here

page.tsx went from **692 lines to 15**. Everything it used to do now lives in src/components/dashboard/, which is the difference between behaviour that can be unit tested and behaviour that can only be tested through a request.

dashboard.css grew by 45 lines - the installed-models panel and the cold-start row in the results needed styling.

The stylesheet is entirely scoped under .epd. That scoping is why src/app/globals.css could shrink from 418 lines to 27: the dashboard's design system stopped leaking into the home page and the vision dashboard.

Files

File	What it is
page.tsx	16 lines. Metadata and a mount point
dashboard.css	614 lines. The module-scoped design system. Light and dark, reduced-motion, focus-visible, and a colour-vision-deficiency validated provider palette in which colour follows the provider entity, never rank or state
loading.tsx	16 lines. The App Router's route-level loading convention
error.tsx	30 lines. The route-level error boundary - which is why the team's three separate loading and error-boundary components were not needed

Connected folders

- src/components/dashboard/ - where the 692 lines went.
- docs/dashboard/ - the module guide.
- src/app/ - why globals.css is 27 lines.

src/app/evidence/ - the evidence page

Work package: DevOps.

What this folder is

The /evidence route: everything in evidence/ rendered in the browser, so a reviewer can read the artefacts without opening a terminal.

What happened here

New. Two things about it are deliberate.

It leads with the limits. Every artefact is introduced by its own `what_this_proves` and `what_this_does_not_prove` fields before a single number appears. An evidence page that opened with a score would be a marketing page.

Its file reads use literal path segments. The first version built paths with a dynamic `path.join(process.cwd(), root)` inside a loop, which defeats Next's file tracing - the tracer cannot see which files are needed, so it conservatively traced the entire project into the standalone bundle. Rewritten as two explicit calls with literal segments.

Files

File	What it is
page.tsx	286 lines. Reads evidence/benchmark/ and evidence/evaluation/ at request time and renders each artefact with its claims and its caveats

Connected folders

- evidence/ - what this page reads.
- evidence/evaluation/ - the ten-case matrix it renders.
- scripts/ - what writes the files.

Referenced from

These folders point here. Each link below resolves in both directions.

- evidence/benchmark/
- evidence/vision-benchmark/
- src/app/

src/app/vision-benchmark/ - the vision dashboard route

Work package: vision benchmark.

What this folder is

The /vision-benchmark route: the image-classification workload, its results, and the bring-your-own-dataset panel.

What happened here

Grew from 73 to 244 lines, almost all of it to host the dataset upload. That feature is the reason the page has a client component in it at all.

The design rule the page has to respect: an uploaded image **never reaches the EdgePilot server**. It is resized and hashed in the tab, and classified by talking from the tab straight to the user's own Ollama on localhost. So the page mounts a browser image processor and a browser provider rather than posting files to an endpoint.

Files

File	What it is
page.tsx	245 lines. The workload description, the results view, and the mount point for the dataset upload

Connected folders

- src/components/vision/ - DatasetUpload.tsx, the upload panel.
- src/modules/vision-benchmark/ - the module.
- src/components/dashboard/ - VisionHandoff.tsx, how a user arrives here from the benchmark dashboard.
- docs/vision-benchmark/ - the workload specification.

Referenced from

These folders point here. Each link below resolves in both directions.

- src/app/

src/app/api/v1/ - the versioned API

Work package: integration and API surface, with individual routes belonging to their modules.

What this folder is

Nine endpoints. Every one of them follows the same envelope - { success, data } on the way out, { success: false, error, details } on failure - because the dashboard renders exactly one error shape and any deviation costs a special case in the UI.

None of these files constructs an adapter, reads an environment variable, or touches Prisma directly. They ask `src/modules/benchmark/infrastructure/container.ts` for a wired use case and get one. That is what keeps a route to parse, dispatch and choose a status code.

What happened here

Four endpoints are new - comparisons, local-runtime, session-log and session-log/share. One was removed - devices. And one, workloads, was fixed from a scaffold that never called the database at all.

Ownership across all of them moved from a single hardcoded shared user to a per-session user derived from the `x-edgepilot-session` header. Under the old behaviour a hosted deployment would have handed every visitor every other visitor's rows.

Subfolders

Endpoint	What it does
benchmarks/	POST run and record; GET list for one owner
comparisons/	POST compare two or more models
local-runtime/	GET is Ollama running, and what does it have
providers/	GET the provider catalogue and what is configured
readiness/	GET the readiness score for one benchmark
session-log/	GET export, DELETE discard
vision-benchmarks/	GET/POST the vision workload
workloads/	POST create a workload row; GET list

Connected folders

- `src/modules/benchmark/infrastructure/` - the composition root every route calls.
- `src/lib/` - `sessionOwner.ts`, how these routes decide who owns a row.
- `docs/internal/` - the wire contract.
- `tests/benchmark/` - `benchmarks-route.test.ts` drives the main route with the container mocked.

Referenced from

These folders point here. Each link below resolves in both directions.

- `src/app/api/`

- src/components/dashboard/
- src/modules/
- src/shared/

src/app/api/v1/benchmarks/ - run and record a benchmark

Work package: benchmark and provider layer.

What this folder is

POST /api/v1/benchmarks runs a benchmark and records it. GET lists recorded benchmarks for one owner.

What happened here

The response envelope is unchanged from the scaffold, deliberately - the dashboard is written against it, and everything new was added *inside* data rather than beside it.

This route is server-side only and must stay that way: it is the only thing in the request path that can see GEMINI_API_KEY and GROQ_API_KEY.

The measurement it now returns is a different thing from what it used to return: a real hardware fit instead of the constant 50, a discarded cold-start iteration reported separately, and a readiness score with privacy taken out of the average.

Files

File	What it is
route.ts	174 lines. POST and GET, plus dynamic and maxDuration. maxDuration matters - a cold local model can take over half a minute to answer its first request

Connected folders

- src/modules/benchmark/application/use-cases/ - RunBenchmark, what this dispatches to.
- src/modules/benchmark/infrastructure/ - container.ts, where the wired use case comes from.
- tests/benchmark/ - benchmarks-route.test.ts.
- docs/internal/ - benchmark-api.md.

Referenced from

These folders point here. Each link below resolves in both directions.

- src/app/api/v1/
- src/modules/benchmark/

src/app/api/v1/comparisons/ - compare two or more models

Work package: benchmark and provider layer.

What this folder is

POST /api/v1/comparisons. New in this version.

What happened here

The response is deliberately three separate things rather than one verdict:

- `plan` - what was decided *before* anything ran: whether the entrants are comparable at all, whether they had to run one after another, and why. A reader who disagrees with the conclusion can check the method first.
- `outcomes` - each run in full and unchanged, so nothing is available only in summarised form.
- `report` - per-dimension winners, each marked established or not.

Nothing is written to the database. A comparison analyses runs; it is not a new kind of record, and making it depend on a workload row would tie a read-only question to the write path.

There is **no UI for this endpoint yet**. It works and is tested; nothing in the dashboard calls it.

Files

File	What it is
<code>route.ts</code>	290 lines. POST, returning plan, outcomes and report

Connected folders

- `src/modules/benchmark/application/use-cases/` - `RunComparison`.
- `src/modules/benchmark/core/services/` - `ComparisonPlanner` and `ComparisonReport`, the two files that make the verdict defensible.
- `tests/benchmark/` - `comparison.test.ts`, `run-comparison.test.ts`.

Referenced from

These folders point here. Each link below resolves in both directions.

- `src/app/api/v1/`
- `src/modules/benchmark/`

src/app/api/v1/local-runtime/

src/app/api/v1/local-runtime/ - is Ollama running

Work package: benchmark and provider layer.

What this folder is

GET /api/v1/local-runtime. New in this version.

What happened here

It answers a question the application previously answered too late. `OllamaProvider.isConfigured()` reports only whether a host string is set - by design, because whether anything is *listening* is a runtime question. The consequence was that a user with Ollama stopped, or on the wrong port, found out from a failed benchmark rather than from the screen where they picked the provider.

Two fields carry the design. `ok` is the single value a caller needs to decide whether to enable a Run button. `remedy` is what to show when `ok` is false - never a bare failure, always the command that fixes it.

No credential is involved and no prompt is sent: this reads a version string and a model list.

Files

File	What it is
route.ts	73 lines. GET, returning runtime state, installed models, and a remedy when something is wrong

Connected folders

- src/modules/benchmark/infrastructure/ - `OllamaCatalog.ts`, which does the work.
- src/components/dashboard/ - `InstalledModels.tsx`, which renders it.
- tests/benchmark/ - `ollama-catalog.test.ts`.

Referenced from

These folders point here. Each link below resolves in both directions.

- src/app/api/v1/
- src/modules/benchmark/

src/app/api/v1/providers/ - the provider catalogue

Work package: benchmark and provider layer.

What this folder is

GET /api/v1/providers - which providers exist, and whether each is usable on this server right now.

What happened here

The behaviour worth knowing is what it does when the database is **not** reachable. It still answers, from the registry alone, with `provider_id: null` and a message explaining why. The dashboard can still render the list and say which providers are configured. Degrading to a useful answer beats returning a 500 for a question that does not strictly need the database.

No credential, and no fragment of one, appears in the response.

Files

File	What it is
route.ts	92 lines. GET, returning the catalogue with <code>is_configured</code> , <code>configuration_hint</code> , database availability, and server configuration warnings

Connected folders

- src/modules/benchmark/infrastructure/providers/ - `ProviderRegistry.ts`.
- prisma/ - `seed.ts` is what puts the catalogue rows there.
- src/components/dashboard/ - `ProviderPanel.tsx`.

Referenced from

These folders point here. Each link below resolves in both directions.

- src/app/api/v1/
- src/modules/benchmark/

src/app/api/v1/readiness/

src/app/api/v1/readiness/ - readiness scores

Work package: benchmark and provider layer.

What this folder is

A container for the dynamic segment below it.

Subfolders

- [id]/ (FOLDER.md) - GET /api/v1/readiness/[id].

Connected folders

- src/app/api/v1/ - the rest of the API.
- src/modules/benchmark/core/services/ - ReadinessCalculator.ts, which produces what this reads back.

src/app/api/v1/session-log/ - export or discard the session record

Work package: benchmark and provider layer.

What this folder is

GET /api/v1/session-log exports this session's activity as JSON. DELETE discards it. New in this version.

What happened here

The session is identified by the x-edgepilot-session header, which the client generates. There is no account, no cookie and no server-assigned identity: a caller that stops sending the header simply has no log.

GET returns a **downloadable document** rather than an API envelope, because the point of it is to be saved and attached to something. The disclosure block travels inside the file, so a copy that changes hands still states what was and was not recorded - which matters, because prompt content is never in it.

There is **no UI for this endpoint yet**.

Files

File	What it is
route.ts	85 lines. GET export, DELETE discard

Subfolders

- share/ - the consent-gated upload.

Connected folders

- src/core/logging/ - SessionLog.ts and the store.
- src/components/dashboard/ - session.ts, which generates the header value.
- tests/core/ - session-log.test.ts.

Referenced from

These folders point here. Each link below resolves in both directions.

- src/app/api/v1/
- src/core/

src/app/api/v1/vision-benchmarks/ - the vision workload endpoint

Work package: vision benchmark.

What this folder is

GET and POST /api/v1/vision-benchmarks - run the image-classification workload server-side, and list what has been run.

What happened here

The route sets runtime explicitly, because the server-side image processor uses sharp, a native module that cannot run on the edge runtime.

Its role narrowed rather than grew. It is now the *server* path for the vision workload - used for the controlled dataset. When a user brings their own images, nothing goes through this endpoint at all: the browser prepares the images and talks straight to the user's own Ollama, so the photographs never leave the machine.

Files

File	What it is
route.ts	128 lines. GET and POST, with dynamic and runtime set

Connected folders

- src/modules/vision-benchmark/ - the module.
- src/modules/vision-benchmark/infrastructure/ - run-service.ts and the sharp processor.
- src/components/vision/ - the browser path that deliberately bypasses this route.
- tests/vision-benchmark/ - vision-api.test.ts.

Referenced from

These folders point here. Each link below resolves in both directions.

- src/app/api/v1/

src/app/api/v1/workloads/ - workload rows

Work package: integration and API surface.

What this folder is

POST /api/v1/workloads creates a workload row and returns its real uuid. GET lists the workloads belonging to the caller.

What happened here

This is the single most consequential fix in the project. The committed version was a scaffold: it returned the literal string `temp-workload-id` and never called Prisma. So nothing the dashboard saved ever reached the database, and every benchmark run afterwards failed with `404 workload not found`. The dashboard had grown a workaround for it - detect the placeholder id, warn the user the row was not saved, and offer a field to paste a real database uuid into by hand.

The route went from 53 lines to 126 and now writes real rows. Ownership comes from the caller's session id, so a row belongs to whoever created it without anyone signing up - and without a uuid ever being shown to, or typed by, a user. The dashboard's paste-a-uuid fallback and the field that went with it are gone.

The original scaffold is kept beside the route as `route.ts.scaffold.bak` and is referenced **by name** in the new file's header, which is why it is explicitly un-ignored in `.gitignore` while every other `.bak` is ignored.

Files

File	What it is
<code>route.ts</code>	126 lines. POST and GET, both scoped to the caller's session
<code>route.ts.scaffold.bak</code>	The original 53-line stub, kept as the reference the docblock points at

Connected folders

- `src/lib/` - `sessionOwner.ts`, where the owner comes from.
- `src/components/dashboard/` - `SetupPanel.tsx`, which lost 379 lines' worth of workaround because of this.
- `scripts/dev/` - `seed-demo-rows.ts`, which existed because this endpoint did not persist.
- `prisma/` - the workloads table.

Referenced from

These folders point here. Each link below resolves in both directions.

- `src/app/api/v1/`
- `src/shared/types/`

src/app/api/v1/readiness/[id]/

src/app/api/v1/readiness/[id]/ - one readiness score

Work package: benchmark and provider layer.

What this folder is

GET /api/v1/readiness/[id] - the readiness score recorded for one benchmark.

What happened here

Two details in this route are easy to get wrong and are pinned by comments in the file.

[id] is the **benchmark** id, not the readiness-score id. readiness_scores has a unique benchmark_id, and the caller holds a benchmark id.

The stored limitations array carries the assumption lines the run was scored under, each prefixed ASSUMPTION:. They are split back out here so a client does not need to know about the prefix - **a score must never be read without the caveats it was computed under**.

What the score itself means changed underneath this route: hardwareFit can now be null where it used to always be 50, and privacyScore is gone from the average entirely.

Files

File	What it is
route.ts	74 lines. GET, returning the score, its breakdown, and its assumptions separated from its limitations

Connected folders

- src/modules/benchmark/core/services/ - ReadinessCalculator.ts and HardwareFitAssessor.ts
- prisma/migrations/ - the migration that made hardware_fit nullable.
- src/components/dashboard/ - RunResults.tsx, which renders it.

Referenced from

These folders point here. Each link below resolves in both directions.

- src/app/api/v1/readiness/
- src/modules/benchmark/

src/app/api/v1/session-log/share/ - consent-gated sharing

Work package: benchmark and provider layer.

What this folder is

GET previews exactly what sharing would send. POST sends it, on an explicit confirmation.

What happened here

New, and the two design rules in it are the reason it is a separate route rather than a flag on the export.

The preview is not a description of the payload - it IS the payload. Both verbs call the same builder. Anything else drifts, and a preview that has drifted from what actually gets sent is worse than no preview.

POST **requires** `confirm: true` **in the body**. A request without it is refused and returns the preview instead, so "share" can never happen as a side effect of some other call. The consent wording the user agreed to is stored on the row, rather than living only in whatever the interface happened to say that day.

There is **no UI for this endpoint yet**.

Files

File	What it is
route.ts	170 lines. GET preview, POST send-on-confirmation

Connected folders

- src/app/api/v1/session-log/ - the export it shares.
- src/core/logging/ - buildSharePayload and SHARE_CONSENT_STATEMENT.
- prisma/migrations/ - the migration that created shared_findings.

src/ - the application

Work package: shared, by subfolder.

What this folder is

Everything next build compiles. Five top-level divisions, and the division is the architecture:

- app/ - HTTP. Routes and pages, nothing else.
- components/ - React. What the user sees.
- modules/ - the two domain modules, each with its own core / application / infrastructure layers.
- core/, lib/, shared/ - things genuinely used by more than one module.

What happened here

Two structural simplifications, both of them removals.

src/data/ was deleted. It was a second data-access path sitting beside src/modules/*/infrastructure/repositories/, which meant the project had two answers to "where does database access live" - and two answers is the same as none.

src/core/rate-limit/ and src/core/security/ were deleted. They guarded endpoints that have no authentication and no public URL. Security machinery that protects nothing still has to be read, maintained and reasoned about by everyone who comes after.

src/core/logging/ gained SessionLog.ts, which replaced a generic logger.ts with something that has an actual policy - a redaction rule, an export format, and opt-in behaviour.

Subfolders

Subfolder	What it holds
app/	Routes and pages - the HTTP surface
components/	React components
modules/	The benchmark and vision-benchmark domain modules
core/	Cross-cutting concerns - currently the session log
lib/	The Prisma client and session ownership
shared/	Types shared across module boundaries

Connected folders

- tests/ - mirrors this tree.
- prisma/ - the schema the repositories map onto.
- docs/internal/ - architecture.md describes these boundaries in full.

Referenced from

These folders point here. Each link below resolves in both directions.

